



UNIVERSITÀ DEGLI STUDI DELL'INSUBRIA

Dipartimento di Scienze Teoriche e Applicate



LABORATORIO INTERDISCIPLINARE A

Progetto CineMax, a.a. 2025/2026

MANUALE

TECNICO

A CURA DI

NOME E COGNOME	MATRICOLA	SEDE
De Agostini Davide	766294	CO
d'Elia Luigi	765969	CO
Saeed Ahsan	767241	CO
Zecchini Martina	765842	CO

Indice

1. Installazione	
1.1 Requisiti di sistema	
1.2 Setup ambiente.....	
1.3 Installazione del programma	
2. Esecuzione ed uso	
2.1 Setup e lancio del programma	
2.2 Panoramica delle funzionalità per ruolo.....	
2.3 Data set di test.....	
3. Architettura dell'applicazione	
3.1 Organizzazione in package e classi	
3.2 Scelte architetturali principali.....	
3.3 Pattern Archivio / Gestore / Schermata	
4. Strutture dati	
4.1 Array dinamici auto-espandibili.....	
4.2 Classi di dominio	
4.3 Chiave composta al posto dell'id numerico.....	
5. Scelte algoritmiche	
5.1 Ordinamento cronologico (Selection Sort)	
5.2 Ricerca con criteri combinabili in AND	
5.3 Generazione di codici prenotazione univoci.....	
5.4 Rilevamento delle sovrapposizioni tra proiezioni.....	
6. Persistenza su file	
6.1 Formato dei file e percorsi.....	
6.2 Parsing CSV con gestione dei campi quotati.....	
6.3 Gestione robusta della riga di intestazione	
6.4 Sicurezza delle password (cifatura SHA-256)	
7. Javadoc	
8. Limiti della soluzione sviluppata	
9. Sitografia / Bibliografia	

1. Installazione

1.1 Requisiti di sistema

CineMax è un'applicazione Java a interfaccia testuale (TUI), pensata per essere multiplatforma. I requisiti minimi sono:

- ❖ Java Development Kit (JDK) versione 17 o successiva (l'applicazione usa esclusivamente API della libreria standard: `java.time`, `java.io`, `java.security`, `java.util.Scanner`);
- ❖ un terminale in grado di leggere/scrivere testo in codifica UTF-8, dato che nomi di film e clienti possono contenere caratteri accentati;
- ❖ nessuna dipendenza esterna: il progetto non richiede librerie di terze parti (non è presente alcuna cartella `lib` con contenuti obbligatori);
- ❖ circa 1 MB di spazio su disco per sorgenti, eseguibile e file di dati.

1.2 Setup ambiente

Non è necessario installare un application server o un database: la persistenza è realizzata tramite semplici file CSV in una cartella `data/` relativa alla working directory del processo. È sufficiente:

- ❖ installare il JDK (es. Eclipse Temurin, Oracle JDK) e verificare la sua presenza con il comando:

TERMINALE

```
$ java -version
$ javac -version
```

- ❖ clonare o scaricare il repository del progetto, mantenendo intatta la struttura delle cartelle (in particolare `src/`, `data/` e `bin/`), come richiesto dalle specifiche di consegna.

1.3 Installazione del programma

La compilazione avviene da riga di comando, posizionandosi nella cartella `src` del repository (che contiene la cartella del package `cinemax`):

TERMINALE

```
$ javac -d ../bin -encoding UTF-8 cinemax/*.java
```

Il flag `-encoding UTF-8` è importante: senza di esso, su ambienti Windows con codifica di default diversa, i caratteri accentati nei file CSV potrebbero essere letti o scritti in modo scorretto.

In alternativa, per generare direttamente l'eseguibile `.jar` richiesto dalla consegna:

TERMINALE

```
$ cd bin
$ jar cfe CineMax.jar cinemax.CineMax cinemax/*.class
```

Il manifest generato con l'opzione `-e` imposta automaticamente `cinemax.CineMax` come classe con il metodo `main`, in conformità al punto 4 delle specifiche di progetto («Il metodo `main` [...] deve essere contenuto nella classe di nome `CineMax` del package `cinemax`»).

2. Esecuzione ed uso

2.1 Setup e lancio del programma

Il programma va eseguito dalla cartella che contiene la sottocartella `data/` (poiché `CineMax.java` usa il percorso relativo «`data`»), altrimenti gli archivi non trovano i file CSV al primo avvio:

TERMINALE

```
$ java -cp bin cinemax.CineMax
# oppure, con il .jar:
$ java -jar bin/CineMax.jar
```

Al primo avvio, `ArchivioAccount`, `ArchivioSpettacoli` e `ArchivioBiglietti` tentano il caricamento rispettivamente di `data/utenti.csv`, `data/proiezioni.csv` e `data/prenotazioni.csv`. Se un file non esiste, l'applicazione non si blocca: stampa un avviso e prosegue con un archivio vuoto (`proiezioni.csv` e `prenotazioni.csv` verranno creati automaticamente al primo salvataggio; `utenti.csv`, invece, deve essere consegnato già popolato, come richiesto dalle specifiche, con almeno 2 account OPERATORE e 5 account CASSIERE).

2.2 Panoramica delle funzionalità per ruolo

Dal punto di vista tecnico, ogni ruolo applicativo corrisponde a una schermata dedicata (classe `Schermata*`), instradata da `SchermataIniziale` in base al valore del campo `ruolo` dell'Account autenticato:

Ruolo (Account)	Schermata	Funzionalità principali
n/d (ospite)	<code>SchermataOspite</code>	Ricerca proiezioni (multi-criterio), visualizzazione dettagli, lista proiezioni future
SPETTATORE	<code>SchermataSpettatore</code>	Ricerca proiezioni, creazione/modifica/eliminazione prenotazioni, visualizzazione prenotazioni personali
OPERATORE	<code>SchermataOperatore</code>	Aggiunta/modifica/eliminazione proiezioni, visualizzazione palinsesto completo
CASSIERE	<code>SchermataCassiere</code>	Ricerca prenotazioni, prenotazioni della giornata, ricerca per nome/cognome cliente

Per la descrizione passo-passo dell'uso delle singole funzionalità, corredata di esempi di interazione a terminale, si rimanda al Manuale Utente.

2.3 Data set di test

Il file `data/proiezioni.csv` consegnato dal docente (`proiezioni.csv`, rinominato per coerenza) viene caricato correttamente grazie alla gestione tollerante della riga di intestazione descritta al punto 6.3: la prima riga viene interpretata come dato e, se il parsing fallisce (perché contiene testo al posto di numeri o date), viene scartata silenziosamente.

Il file `data/utenti.csv` di test contiene 2 account OPERATORE e 5 account CASSIERE con password comune «cinemax2026», già cifrata in SHA-256 nel file. Il file `data/prenotazioni.csv` di test contiene solo la riga di intestazione: le prenotazioni vengono generate dall'applicazione durante l'uso (registrazione di un cliente + creazione di una prenotazione).

Il flusso di test end-to-end eseguito prima della consegna copre: registrazione nuovo cliente, login con i tre ruoli, ricerca proiezioni con criteri singoli e combinati, creazione di una prenotazione con verifica dei posti disponibili, tentativo di prenotazione oltre la capienza (rifiutato), modifica e cancellazione di una prenotazione, aggiunta di una proiezione sovrapposta a una esistente (rifiutata), e round-trip di lettura/scrittura di tutti e tre i file CSV.

3. Architettura dell'applicazione

3.1 Organizzazione in package e classi

L'intero progetto è contenuto in un unico package `cinemax` (nessun sotto-package), scelta deliberata per mantenere la struttura piatta e facilmente navigabile in un progetto di dimensioni contenute (20 classi, `CineMax.java` con il main incluso). Le classi si raggruppano concettualmente in cinque livelli, più il punto di ingresso:

- ❖ **Modello di dominio:** `Account`, `Pellicola`, `Spettacolo`, `Biglietto`: semplici classi con campi privati, costruttore, getter/setter e `toString()`;
- ❖ **Persistenza:** `ArchivioAccount`, `ArchivioSpettacoli`, `ArchivioBiglietti`: caricamento/salvataggio su CSV e accesso in memoria tramite array;
- ❖ **Logica applicativa:** `GestoreAccessi`, `GestoreSpettacoli`, `GestoreBiglietti`, `MotoreRicerca`: regole di business (validazioni, calcoli, ricerche);
- ❖ **Presentazione:** `SchermataIniziale`, `SchermataOspite`, `SchermataSpettatore`, `SchermataOperatore`, `SchermataCassiere`: interazione testuale con l'utente, un menu per ruolo;
- ❖ **Utilità trasversali:** `CsvUtile` (parsing/composizione righe CSV), `Cifratura` (hashing password), `GeneratoreCodice` (codici prenotazione univoci).

Il punto di ingresso è `CineMax.java`, che si limita a istanziare archivi, gestori e la schermata iniziale, e a passarli in cascata via constructor injection: nessun singleton, nessuna dipendenza statica nascosta tra le classi di logica.

3.2 Scelte architetturali principali

Le scelte architetturali derivano direttamente dai vincoli del corso (RAM/pseudocodice, strutture dati e algoritmi visti a lezione) più che da un uso «enterprise» di Java:

- ❖ Nessuna collection: si usano esclusivamente array e campi primitivi, coerentemente con l'obiettivo didattico del laboratorio di mettere in pratica strutture dati implementate manualmente, invece di delegare a `java.util.ArrayList` e simili;
- ❖ `java.time` al posto di classi custom: `LocalDate` e `LocalDateTime` della libreria standard sostituiscono una eventuale classe `DataOra` scritta a mano, evitando di reinventare calcoli di validità e formattazione già risolti correttamente dalla libreria;
- ❖ Gestione delle eccezioni sempre interna (try-catch, mai throws in firma): ogni metodo pubblico intercetta le proprie eccezioni e restituisce un valore sentinella (null, false, array vuoto) accompagnato da un messaggio su `System.out`, cosicché le classi `Schermata*` non debbano mai gestire eccezioni checked provenienti dagli strati sottostanti;
- ❖ Nessun id numerico surrogato: proiezioni e prenotazioni non hanno un campo id autoincrementale. Le proiezioni sono identificate dalla chiave composta (titolo, data/ora; si veda il punto 4.3), mentre le prenotazioni usano un codice alfanumerico generato ad hoc (punto 5.3);
- ❖ Separazione tra ricerca e logica di dominio: `MotoreRicerca` è una classe a parte rispetto ai `Gestore*`, perché i criteri di ricerca (parziali, intervalli, combinazioni in AND) sono trasversali a più schermate (ospite, cliente, cassiere) e altrimenti sarebbero stati duplicati in ciascuna.

3.3 Pattern Archivio / Gestore / Schermata

Il progetto segue costantemente una separazione a tre livelli per ciascuna entità (proiezioni, prenotazioni, account):

- ❖ Archivio*: incapsula la persistenza su file e l'accesso in memoria (array + contatore di occupazione); espone operazioni CRUD di basso livello (`elencoTutti`, `trovaPer...`, `aggiungi`, `rimuovi`) senza conoscere alcuna regola di business;
- ❖ Gestore*: applica le regole applicative (validazioni, controlli di sovrapposizione, calcolo posti liberi, vincoli su modifica/eliminazione) usando uno o più Archivio* come collaboratori;
- ❖ Schermata*: gestisce l'interazione a terminale (lettura input da Scanner, stampa dei menu), delegando ogni operazione ai Gestore* e non contenendo essa stessa alcuna regola di business.

Questa separazione permette, ad esempio, di riutilizzare `GestoreSpettacoli.postiLiberi()` sia in `SchermataOspite` (per mostrare i posti liberi durante la ricerca) sia in `SchermataSpettatore` (per verificare la disponibilità prima di una prenotazione), senza duplicare la formula «capienza sala meno posti occupati».

Esempio rappresentativo (`GestoreSpettacoli.java`) del confine tra logica applicativa e persistenza: il controllo di sovrapposizione è interamente nel Gestore, mentre la lettura dei dati grezzi è delegata all'Archivio:

GESTORESPETTACOLI.JAVA

```
private boolean siSovrappone(Spettacolo candidata) {
    LocalDateTime inizioCandidata = candidata.getDataOra();
    LocalDateTime fineCandidata = candidata.getDataOraFine();
    Spettacolo[] esistenti = archivioSpettacoli.elencoTutti();
    for (int i = 0; i < esistenti.length; i++) {
        Spettacolo altra = esistenti[i];
        LocalDateTime inizioAltra = altra.getDataOra();
        LocalDateTime fineAltra = altra.getDataOraFine();
        boolean sovrapposte = inizioCandidata.isBefore(fineAltra)
            && inizioAltra.isBefore(fineCandidata);
        if (sovrapposte) {
            return true;
        }
    }
    return false;
}
```

4. Strutture dati

4.1 Array dinamici auto-espandibili

Ogni Archivio* mantiene i propri dati in un array privato (Account[], Spettacolo[], Biglietto[]) inizializzato con capacità 10 e un contatore quantità che tiene traccia degli elementi effettivamente occupati (pattern equivalente a una implementazione manuale di una lista dinamica). Quando l'array è pieno, viene raddoppiato:

ARCHIVIOSPETTACOLI.JAVA

```
private void aggiungiInMemoria(Spettacolo spettacolo) {
    if (quantita == elenco.length) {
        Spettacolo[] nuovoArray = new Spettacolo[elenco.length * 2];
        for (int i = 0; i < elenco.length; i++) {
            nuovoArray[i] = elenco[i];
        }
        elenco = nuovoArray;
    }
    elenco[quantita] = spettacolo;
    quantita++;
}
```

Il raddoppio ammortizza il costo delle inserzioni successive (costo ammortizzato $O(1)$ per inserimento, a fronte di un costo $O(n)$ solo nelle inserzioni che innescano la copia). La rimozione (ArchivioBiglietti.rimuovi, ArchivioSpettacoli.rimuovi) scorre l'array a partire dall'elemento trovato e trasla di una posizione tutti gli elementi successivi, evitando di lasciare «buchi» e mantenendo l'invariante che gli elementi occupati siano sempre le prime quantità posizioni.

Ogni metodo di lettura pubblico (elencoTutti) restituisce una copia dell'array, mai il riferimento interno: questo impedisce a codice esterno di modificare la struttura dati dell'Archivio* aggirando i controlli dei Gestore*.

4.2 Classi di dominio

Le quattro classi di dominio sono POJO essenziali (campi privati, costruttore, getter/setter, toString), senza logica di business al loro interno; quella risiede nei *Gestore**, coerentemente con la separazione descritta al punto 3.3:

- ❖ *Account*: nome, cognome, username, password cifrata, data di nascita (facoltativa, può essere null), domicilio, ruolo. Il ruolo è rappresentato con tre costanti String (SPETTATORE, OPERATORE, CASSIERE) invece che con tre sottoclassi, perché i tre ruoli non differiscono per struttura dati ma solo per operazioni permesse, già gestite altrove dalle classi *Gestore*/Schermata**;
- ❖ *Pellicola*: titolo, genere, regista, anno, durata in minuti, età minima;
- ❖ *Spettacolo*: una *Pellicola*, una *LocalDateTime*, un prezzo; espone *getDataOraFine()* (calcolata sommando la durata del film all'orario di inizio) e il metodo *corrispondeA(titolo, dataOra)* usato per la chiave composta (punto 4.3);
- ❖ *Biglietto*: codice, username del cliente, titolo del film e data/ora della proiezione (denormalizzati anziché referenziati con un puntatore a *Spettacolo*, per restare persistibili in modo semplice su una riga CSV), numero di posti, costo unitario.

4.3 Chiave composta al posto dell'id numerico

Le proiezioni non hanno un id numerico: sono identificate dalla coppia (titolo del film, data/ora), confrontata senza distinzione tra maiuscole e minuscole sul titolo. Questa scelta funziona in modo sicuro perché *GestoreSpettacoli.modificaProiezione()* ed *eliminaProiezione()* sono bloccate ogni volta che esistono prenotazioni collegate a quella proiezione: di conseguenza, una proiezione referenziata da almeno una prenotazione non può più cambiare titolo o data/ora, e il riferimento salvato nel *Biglietto* resta sempre valido.

SPETTACOLO.JAVA

```
public boolean corrispondeA(String titolo, LocalDateTime dataOra) {
    if (titolo == null || dataOra == null) {
        return false;
    }
    return film.getTitolo().equalsIgnoreCase(titolo)
        && this.dataOra.equals(dataOra);
}
```

Spettacolo ridefinisce *equals()/hashCode()* sulla stessa chiave composta, per coerenza con *corrispondeA()* qualora la classe venisse in futuro inserita in strutture basate su hashing. Le prenotazioni, invece, hanno un vero identificatore surrogato (il codice alfanumerico generato da *GeneratoreCodice*), perché a differenza delle proiezioni non esiste per esse un attributo naturale stabile e univoco nel tempo.

5. Scelte algoritmiche

5.1 Ordinamento cronologico (Selection Sort)

Sia `GestoreSpettacoli.elencoPalinsesto()` sia `GestoreBiglietti.prenotazioniDiOggi()` ordinano un array copia con un selection sort in place, scelto per la sua semplicità didattica rispetto a un merge sort o un quick sort, ed è comunque adeguato viste le dimensioni tipiche di un palinsesto di un cinema monosala (poche migliaia di proiezioni al più, complessità $O(n^2)$ accettabile):

GESTORESPETTACOLI.JAVA

```
public Spettacolo[] elencoPalinsesto() {
    Spettacolo[] elenco = archivioSpettacoli.elencoTutti();
    for (int i = 0; i < elenco.length - 1; i++) {
        int indiceMinimo = i;
        for (int j = i + 1; j < elenco.length; j++) {
            if (elenco[j].getDataOra().isBefore(elenco[indiceMinimo].getDataOra())) {
                indiceMinimo = j;
            }
        }
        if (indiceMinimo != i) {
            Spettacolo temporaneo = elenco[i];
            elenco[i] = elenco[indiceMinimo];
            elenco[indiceMinimo] = temporaneo;
        }
    }
    return elenco;
}
```

L'ordinamento avviene sempre su una copia restituita da `elencoTutti()` (si veda il punto 4.1): l'array interno dell'`Archivio*` non viene mai alterato da un'operazione di sola lettura.

5.2 Ricerca con criteri combinabili in AND

`MotoreRicerca.cercaProiezione()` e `cercaPrenotazione()` implementano una ricerca lineare $O(n)$ sull'intero archivio, applicando in sequenza ogni criterio non nullo con cortocircuito logico (un flag corrisponde booleano che, una volta diventato false, salta i controlli successivi per quell'elemento). Ogni parametro può essere null (o stringa vuota per i titoli) per essere semplicemente ignorato, permettendo di riusare lo stesso metodo sia per ricerche a criterio singolo (es. solo titolo, dalla schermata ospite) sia per ricerche combinate (titolo + genere + intervallo di date + intervallo di prezzo).

Il risultato viene accumulato in un array temporaneo dimensionato come il totale degli elementi in archivio (caso peggiore: tutti corrispondono), poi copiato in un array della dimensione esatta trovata, evitando così sia un `ArrayList` sia una doppia scansione per contare prima i risultati.

5.3 Generazione di codici prenotazione univoci

`GeneratoreCodice` mantiene un contatore statico e compone codici nel formato «PRN-000001», verificando ad ogni generazione che il candidato non sia già presente tra le prenotazioni esistenti (protezione ridondante rispetto al solo contatore, utile nel caso in cui il contatore venga resettato tra un'esecuzione e l'altra del programma, dato che non viene persistito su file):

GENERATORECODICE.JAVA

```
public static String generaCodicePrenotazione(Biglietto[] prenotazioniEsistenti) {
    String candidato;
    boolean giaUsato;
```

```

do {
    candidato = String.format("PRN-%06d", contatore);
    contatore++;
    giaUsato = false;
    for (int i = 0; i < prenotazioniEsistenti.length; i++) {
        if (prenotazioniEsistenti[i].getCodice().equalsIgnoreCase(candidato)) {
            giaUsato = true;
            break;
        }
    }
} while (giaUsato);
return candidato;
}

```

5.4 Rilevamento delle sovrapposizioni tra proiezioni

Il controllo di non sovrapposizione richiesto dalle specifiche (aggiungiProiezione() deve rifiutare una proiezione se si sovrappone a una esistente) è implementato confrontando gli intervalli [inizio, fine) di ciascuna coppia di proiezioni con la classica condizione di intersezione tra intervalli: due intervalli si sovrappongono se e solo se l'inizio del primo precede la fine del secondo e l'inizio del secondo precede la fine del primo (si veda lo snippet al punto 3.3). L'istante di fine di una proiezione è calcolato sommando la durata del film all'orario di inizio (Spettacolo.getDataOraFine()), quindi la sovrapposizione tiene correttamente conto anche di film con durate diverse.

6. Persistenza su file

6.1 Formato dei file e percorsi

Tutti i dati sono persistiti in tre file CSV nella cartella data/, relativa alla working directory del processo:

File	Gestito da	Campi (dopo l'intestazione)
data/proiezioni.csv	ArchivioSpettacoli	data_ora_proiezione, titolo_film, genere, regista, anno, durata_minuti, eta_minima, prezzo_biglietto
data/utenti.csv	ArchivioAccount	nome, cognome, username, password_cifrata, data_nascita, domicilio, ruolo
data/prenotazioni.csv	ArchivioBiglietti	codice, username_cliente, titolo_film, data_ora_spettacolo, numero_posti, costo_unitario

Le date sono lette con il formattatore `FORMATO_LETTURA` («yyyy-MM-dd HH:mm[:ss]»), secondi opzionali, per accettare sia il file fornito dal docente sia i file generati dall'applicazione) e scritte con `FORMATO_SCRITTURA` («yyyy-MM-dd HH:mm»), entrambi definiti come costanti in `Spettacolo` per essere condivisi da tutte le classi che leggono o scrivono date.

Ogni `salvaSuFile()` riscrive interamente il file da zero a partire dall'elenco in memoria (nessun append incrementale): scelta più semplice e meno soggetta a errori di sincronizzazione, adeguata al volume di dati previsto e al vincolo esplicito delle specifiche di non doversi preoccupare di accesso concorrente.

6.2 Parsing CSV con gestione dei campi quotati

`CsvUtile` implementa un parser CSV a mano (nessuna libreria esterna) che rispetta le virgolette: un campo racchiuso tra virgolette può contenere virgole al proprio interno (necessario per i titoli dei film, che talvolta contengono la virgola) e le virgolette raddoppiate al suo interno rappresentano una virgoletta letterale, secondo la convenzione CSV standard.

CSVUTILE.JAVA

```
public static String[] dividiRiga(String riga) {
    String[] campi = new String[10];
    int numeroCampi = 0;
    StringBuilder campoCorrente = new StringBuilder();
    boolean dentroVirgolette = false;
    for (int i = 0; i < riga.length(); i++) {
        char carattere = riga.charAt(i);
        if (carattere == '"') {
            dentroVirgolette = !dentroVirgolette;
        } else if (carattere == ',' && !dentroVirgolette) {
            campi = aggiungiCampo(campi, numeroCampi, campoCorrente.toString());
            numeroCampi++;
            campoCorrente = new StringBuilder();
        } else {
            campoCorrente.append(carattere);
        }
    }
    // ultimo campo, array ridimensionato dinamicamente se necessario
    ...
}
```

Simmetricamente, `componiRiga()` racchiude tra virgolette solo i campi che contengono una virgola o una virgoletta, minimizzando il «rumore» visivo nei file generati e mantenendo la compatibilità con il formato consegnato dal docente.

6.3 Gestione robusta della riga di intestazione

Tutti e tre gli `Archivio*` condividono lo stesso approccio nella lettura: la prima riga del file viene semplicemente tentata come dato. Se il parsing genera un'eccezione (perché la riga contiene testo, es. «titolo_film», al posto di un numero o di una data), l'eccezione viene intercettata e la riga scartata senza stampare alcun avviso, assumendo che si tratti dell'intestazione; se invece un'eventuale riga successiva genera un'eccezione, questa viene segnalata a schermo come riga malformata.

Questa scelta evita di dover distinguere «a priori» se un file ha o meno l'intestazione (il file fornito dal docente ce l'ha, i file generati dall'applicazione pure), e rende il caricamento uniforme per entrambi i casi.

6.4 Sicurezza delle password (cifratura SHA-256)

Le password non sono mai salvate in chiaro. `Cifratura.cifra()` calcola l'impronta SHA-256 del testo in chiaro tramite `java.security.MessageDigest` e la converte in una stringa esadecimale; `Cifratura.corrisponde()` confronta una password inserita dall'utente con il valore cifrato salvato, ricalcolando l'hash e confrontando le due stringhe (mai decifrando quella salvata, operazione impossibile per un hash):

CIFRATURA.JAVA

```
public static String cifra(String testoInChiaro) {
    try {
        MessageDigest algoritmo = MessageDigest.getInstance("SHA-256");
        byte[] risultato = algoritmo.digest(
            testoInChiaro.getBytes(StandardCharsets.UTF_8));
        StringBuilder esadecimale = new StringBuilder();
        for (int i = 0; i < risultato.length; i++) {
            esadecimale.append(String.format("%02x", risultato[i]));
        }
        return esadecimale.toString();
    } catch (NoSuchAlgorithmException erroreCifratura) {
        System.out.println("Errore durante la cifratura della password: "
            + erroreCifratura.getMessage());
        return null;
    }
}
```

La codifica dei byte è fissata esplicitamente a `StandardCharsets.UTF_8` (anziché lasciare Java usare la codifica di default del sistema operativo), per garantire che lo stesso testo produca sempre lo stesso hash indipendentemente dalla piattaforma di esecuzione, requisito di multiplatforma richiesto dalle specifiche di progetto.

7. Javadoc

Tutte le classi, i metodi pubblici e gli attributi rilevanti sono documentati con commenti Javadoc (`/** ... */`), come richiesto dalle specifiche di progetto. In particolare:

- ❖ ogni classe riporta una descrizione dello scopo seguita dai tag `@author` per tutti e quattro i componenti del gruppo, su righe separate;
- ❖ ogni metodo pubblico non banale documenta parametri (`@param`), valore di ritorno (`@return`) dove applicabile, e le eventuali condizioni che determinano un esito negativo (descritte in prosa, dato che il progetto non usa eccezioni checked in firma);
- ❖ i commenti sono scritti con un linguaggio tecnico, coerente con il pubblico di destinazione della Javadoc (sviluppatori).

La documentazione HTML è generata con il tool `javadoc` della JDK e consegnata nella cartella `doc/` del repository, insieme al presente manuale e al Manuale Utente, secondo quanto richiesto dalle specifiche di consegna:

TERMINALE

```
$ javadoc -d ../doc/javadoc -encoding UTF-8 -charset UTF-8 -author cinemax/*.java
```

8. Limiti della soluzione sviluppata

- ❖ Nessuna gestione della concorrenza: come esplicitamente consentito dalle specifiche di progetto, l'applicazione non gestisce accessi concorrenti multi-utente né architetture client/server; è pensata per un solo processo/utente alla volta sullo stesso file di dati;
- ❖ Riscrittura completa del file ad ogni salvataggio: per archivi di dimensioni molto elevate (decine di migliaia di proiezioni o prenotazioni) le prestazioni di `saveToFile()` degraderebbero, non essendo pensato un meccanismo di append incrementale;
- ❖ Capienza sala fissa e cablata: `GestoreSpettacoli.CAPIENZA_SALA` è una costante (200 posti), coerente con le specifiche («piccoli cinema monosala da 200 posti»), ma non configurabile senza ricompilare;
- ❖ Contatore dei codici prenotazione non persistente: essendo un campo statico in memoria, riparte da 1 ad ogni riavvio dell'applicazione; l'unicità è comunque garantita dal controllo esplicito contro le prenotazioni già presenti su file (punto 5.3), ma i codici non sono strettamente crescenti tra un'esecuzione e l'altra;
- ❖ Nessuna validazione avanzata dei dati anagrafici: campi come domicilio o nome non sono validati oltre al controllo di username duplicato in fase di registrazione;
- ❖ Ricerca lineare $O(n)$: adeguata alle dimensioni previste per un cinema monosala, ma non scalerebbe a dataset di proiezioni/prenotazioni molto più grandi senza l'introduzione di strutture indicizzate (es. tabelle hash o alberi di ricerca).

9. Sitografia / Bibliografia

- ❖ Oracle, «How to Write Doc Comments for the Javadoc Tool», Online: <https://www.oracle.com/technical-resources/articles/java/javadoc-tool.html>
- ❖ Oracle, «Java Platform, Standard Edition Documentation» (java.time, java.io, java.security), Online: <https://docs.oracle.com/en/java/javase/17/docs/api/>
- ❖ L. Bozzato, «Introduzione a Git e GitHub», Laboratorio Interdisciplinare A, Università degli Studi dell'Insubria, a.a. 2025/2026
- ❖ G. Meroni, «Documentazione di Progetto» e «Specifiche di Progetto: CineMax», Laboratorio Interdisciplinare A, Università degli Studi dell'Insubria, a.a. 2025/2026